SPLITSMART – GROUP EXPENSE SETTLER

Project Report

Navaneeth A – 2025115151
Mohamed Shaqeeq – 2025115141
Aushadha Kumar - 2025115169

1. Introduction

SplitSmart is a C-based group expense management program. It is used to keep track of expenses when a group of people share money during a trip or any other group activity.

When multiple people pay for different expenses, it can become difficult to calculate who owes money to whom. SplitSmart keeps track of the members, expenses and their balances, and then generates a simplified payment plan.

The project is implemented using C and different data structures such as arrays, linked lists, a debt matrix and two heaps (a max heap and a min heap).

2. Objectives

The main objectives of the project are:

- **To add and manage members in a group.**
- **To record expenses made by different members.**
- **To divide an expense between selected members.**
- **To calculate the net balance of every member.**
- **To display the raw debts between members.**
- **To generate a simplified settlement plan.**
- **To undo the last added expense.**
- **To save the trip details into a file.**
- **To load previously saved trip details.**

3. Features

3.1 Add Members

The user can add members to the group by entering their names. The program supports up to 20 members.

3.2 Add Expenses

For every expense, the user enters:

- Description of the expense
- Amount
- Person who paid
- Members who shared the expense

The expense can either be shared by everyone or only by selected members.

3.3 Expense History

The program stores every expense with its description, amount, payer and the members sharing it. The total number of expenses is shown at the top of the main menu.

3.4 Net Balances

The program calculates the net balance of every member.

A positive balance means that the person has to receive money, while a negative balance means that the person has to pay money.

3.5 Raw Debts

The program also maintains the individual debts between members. This shows how much one member owes another member before the final settlement.

3.6 Settle Up

The settle up option simplifies the payments between members.

The members who have to receive money are stored in one max heap and the members who have to pay are stored in a min heap on their balance, so the person who owes the most is on top. The program then matches the largest creditor with the largest debtor and generates the required payments.

This reduces the number of separate payments required between the members.

3.7 Undo Expense

The last added expense can be removed using the undo option. The balance and debt values are restored by applying the expense in reverse.

3.8 File Handling

The trip information can be saved into a file and loaded later. The saved information contains the trip name, members with their balances, the debt matrix and the expenses. The trip is saved automatically when the program is exited and loaded when it starts.

4. Data Structures Used

Array

Arrays are used for storing the members and their balances. The program has a maximum capacity of 20 members.

Person people[MAX]

An array is also used to store which members are sharing each expense.

Linked List

Expenses are stored using a linked list. Each expense contains the description, amount, payer, members involved and a pointer to the next expense.

The latest expense is inserted at the beginning of the list. This also makes the undo operation easier because the latest expense is always at the top.

Debt Matrix

A two-dimensional array is used to store debts between members.

debt[i][j]

This represents the amount that member i owes to member j.

Heaps

Two heaps are used during settlement:

- Creditor heap (max heap) – contains members who should receive money.
- Debtor heap (min heap on the balance) – contains members who should pay money.

The person with the highest amount is processed first.

5. Working of the Program

The program starts by displaying the main menu whose appearance alone has been designed using Claude.

The user can select different operations such as adding members, adding expenses, viewing balances, settling expenses, saving the data or loading an existing file.

When an expense is added, the amount is divided equally among the selected members. If the amount cannot be divided exactly, the remaining amount is distributed one rupee at a time among the selected members.

The balance of the person who paid is increased by the full expense amount, while the share of each member is deducted from their balance.

After all expenses are entered, the Settle Up option uses the net balances to calculate the final payments.

For example, if one member has to receive Rs 500 and another member has to pay Rs 300, the program generates a payment of Rs 300. The remaining amount is then matched with the next debtor.

6. Algorithm for Expense Addition

1. Get the expense description and amount.
2. Select the member who paid.
3. Select the members who shared the expense.
4. Count the number of members sharing it.
5. Divide the amount among the selected members.
6. Update the payer's balance.
7. Deduct each member's share from their balance.
8. Update the debt matrix.
9. Store the expense in the linked list.

7. Algorithm for Settlement

1. Check the net balance of every member.
2. Add members with positive balances to the creditor heap.
3. Add members with negative balances to the debtor heap.
4. Take the creditor with the highest amount.
5. Take the debtor with the highest amount.
6. Compare their amounts.
7. Generate a payment for the smaller amount.
8. Update the remaining amount.
9. Insert the member back into the heap if some amount is still remaining.
10. Continue until all creditors and debtors are settled.
11. Display the final payment list.

8. Sample Output

```
+------------------------------------------------------------------------+
|          SplitSmart - Group Expenses                                   |
+------------------------------------------------------------------------+
| Trip: My Trip                                                          |
| Members: 3      Expenses: 0                                            |
+------------------------------------------------------------------------+
| 1.  Add a member                                                       |
| 2.  See all members                                                    |
| 3.  Add an expense                                                     |
| 4.  See who is ahead / behind                                          |
| 5.  See who owes whom                                                  |
| 6.  Undo the last expense                                              |
| 7.  Settle up (fewest payments)                                        |
| 8.  Save trip                                                          |
| 9.  Load saved trip                                                    |
| 10. Exit                                                               |
+------------------------------------------------------------------------+
Enter your choice (1-10):
```

After adding expenses, the program displays the balances and settlement information.

Example:

```
--- Who is ahead and who is behind ---
Alice        gets back Rs. 500
Bob          owes Rs. 300
Charlie      owes Rs. 200
```

The Settle Up option then produces a payment list based on these balances.

9. Advantages

- Makes group expense calculation easier.
- Reduces the number of payments required.
- Keeps track of individual expenses.
- Supports undoing the latest expense.
- Data can be saved and loaded using files.
- Demonstrates the practical use of multiple data structures in one application.

10. Limitations

- The program supports a maximum of 20 members.
- Amounts are handled as whole rupees.
- The program is terminal based.
- Invalid user inputs are not handled extensively.
- The settlement generated by the program is a payment plan and does not actually transfer money.

11. Conclusion

SplitSmart is a group expense management program developed using C. The project combines arrays, linked lists, a debt matrix and heaps to solve a practical problem.

The linked list is used to store expenses, while the debt matrix keeps track of individual debts. The heaps are used during settlement to reduce the number of payments required.

The project helped in understanding how data structures can be combined to build a practical application instead of using them separately.